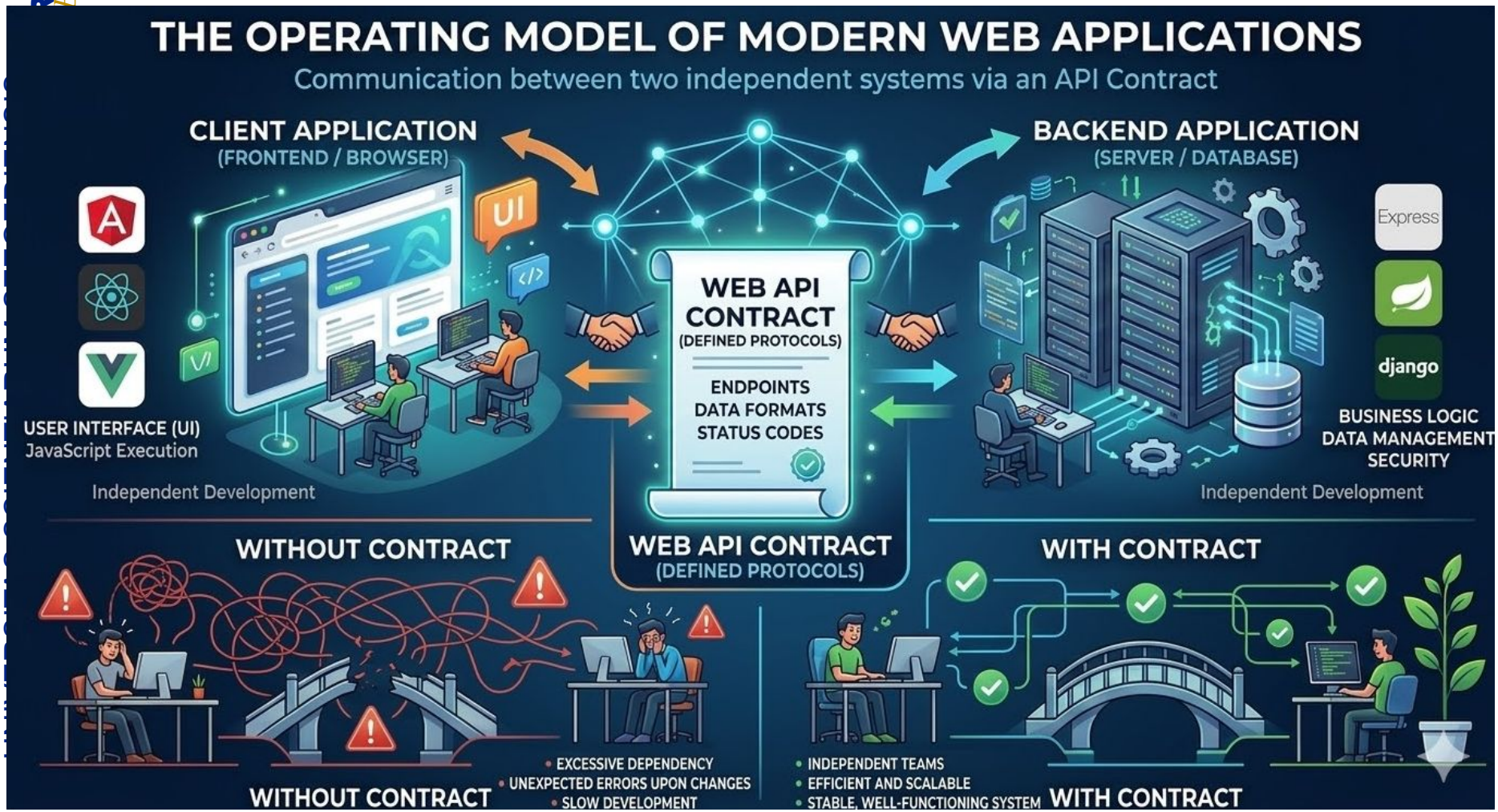
UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Software Architectures

Practice 5

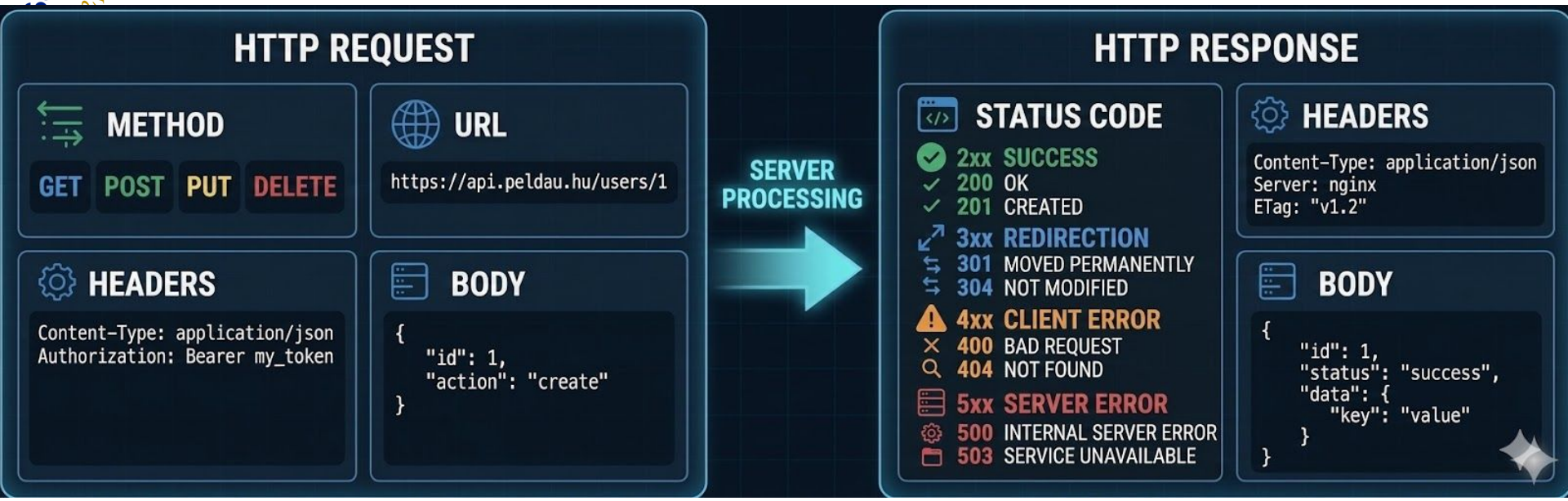
Zoltán Richárd JÁNKI, PhD

The operating model of modern web applications



What is an API?

- API = Application Programming Interface
- A "surface" provided by software, through which other software can communicate with it.
- Web API → HTTP request-response



What is REST?

- REST = Representational State Transfer
- Not a technology, but an architectural style (2000)
- A set of rules for API design
  - Simple
  - Predictable
  - Scalable
- Principles
  - Resource-oriented (identified using nouns)
  - HTTP methods (express verbs or actions)
  - Statelessness (requests must contain all the information necessary for the server)
  - Multiple representations (**JSON**, XML, or HTML)

OpenAPI (Swagger)

- Contract-First Approach: The "agreement" (specification) is created first, and development only begins afterward. This reduces misunderstandings between frontend and backend teams.
- Interactive Documentation: With Swagger UI, the API can be instantly tested and browsed directly from the browser.
  - https://editor.swagger.io/
- Automation: The following can be automatically generated from the specification…
  - Client-side SDKs (in multiple languages)
  - Server-side code (stubs)
  - Automated test cases
- **It's not just documentation, but a common language between developers, testers, and the business side.**

OpenAPI parts

- Metadata
- Endpoint definitions
- Data model

```yaml
openapi: 3.0.3
info:
  title: Recipe Collection API
  description: REST API specification for
  a MEAN stack-based recipe collection web
  application.
  version: 1.0.0
servers:
  - url: http://localhost:3000/api
```

```yaml
paths:
  /users/register:
    post:
      tags: [User]
      summary: User registration
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/
              UserCreate'
            example:
              username: "john_doe"
              email: "john@example.com"
              password: "secretpassword"
      responses:
        '201':
          description: Successful
          registration
          content:
            application/json:
              schema:
                $ref: '#/components/
                schemas/UserResponse'
```

```yaml
components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    UserCreate:
      type: object
      required: [username, email, password]
      properties:
        username:
          type: string
          example: "john_doe"
        email:
          type: string
          example: "john@example.com"
        password:
          type: string
          example: "secretpassword"
    UserLogin:
      type: object
      required: [email, password]
      properties:
        email:
          type: string
          example: "john@example.com"
        password:
          type: string
          example: "secretpassword"
```

UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Philosophies

TWO PHILOSOPHIES:
DESIGN-FIRST vs. CODE-FIRST

DESIGN-FIRST

Write the **OpenAPI spec** before writing any code. The spec becomes the contract that both frontend and backend teams implement against.

BENEFITS:

- Forces you to think before coding
- Catches design issues early
- Enables parallel development

CODE-FIRST

You write the backend code first and generate the **OpenAPI spec** from annotations or decorators.

BENEFITS:

- Feels faster initially
- Spec always matches implementation

OUR RECOMMENDATION & IN PRACTICE

RECOMMENDATION FOR PROJECT COURSES:
DESIGN-FIRST is the better learning experience, because it **forces architectural thinking**. It also matches the course philosophy — conscious, documented decisions before implementation.

IN PRACTICE:
However, many teams use a hybrid approach in professional settings.

Best practices and common mistakes

- Use consistent naming: camelCase or snake_case for properties — pick one and stick with it.
- Be specific with types: don't use string for everything. Use integer for IDs, number for amounts, proper format values.
- Document everything: add description fields to paths, operations, parameters, and schema properties. A spec without descriptions is technically valid but practically useless.
- Define all response codes: don't just define the happy path. Think about what can go wrong.
- Use required accurately: mark fields as required only when they truly are.
- Version your API in the path or header from the start.